KOENIG

step forward



Asynchronous Messaging with JMS

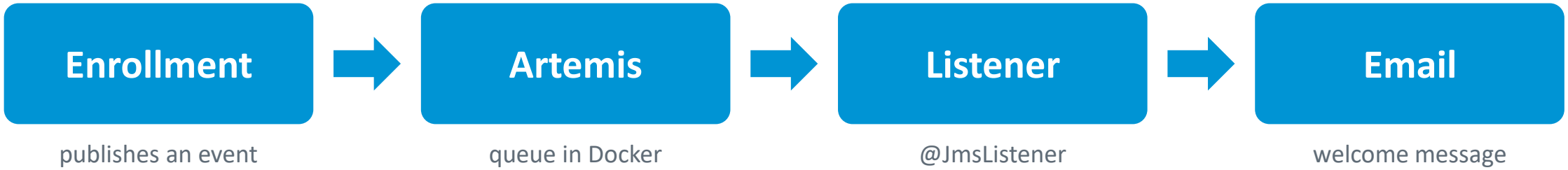
Spring Boot Fundamentals

What You Will Learn

1. Synchronous vs asynchronous
2. JMS concepts
3. Spring JMS and JSON Messages
4. Retry, max delivery, dead letters
5. Azure Service Bus Overview

Demo Scenario: Learnly Enrollment Events

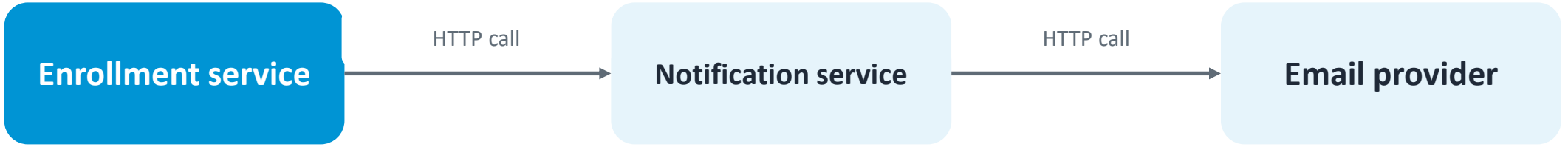
Module 9 moved Learnly's settings out of the code. Now enrolling a student must trigger a welcome email without making the student wait, and the flow must survive failures, slow consumers and duplicates.



We will survive

- **Failures**
- **Dead letters**
- **Duplicates**

Synchronous Communication

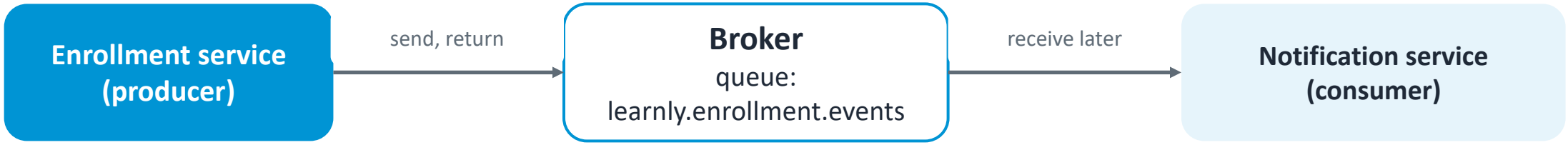


The caller waits for the whole chain to answer

- **Simple:** request, response, done.
- **Tight coupling:** the callee must be up right now.
- **Latency adds up:** slowest service sets the speed.
- **Failures travel back** up the chain.

Like a phone call: both people must be there at the same time

Asynchronous Communication



- **Decoupled in time:** consumer may be down; messages wait.
- **Load levelling:** a burst is absorbed by the queue.
- **Resilience:** failures are retried, not propagated.
- **Trade-off:** eventual consistency, harder to debug.

Like a letter: you post it and go; the reader opens it when ready

Synchronous vs Asynchronous

Synchronous (REST)

Needs an answer now (queries, login, price check).

Simple to reason about and debug.

Caller blocked while waiting.

Both services must be available.

Errors visible immediately.

Asynchronous (messaging)

Work can finish later (emails, reports, events).

Buffers bursts, retries failures.

Caller continues at once.

Services can be down independently.

Errors need monitoring (dead letters).

Most real systems use both: REST to ask, messages to tell

When to Choose Messaging

1 Fire and forget
Notifications, emails, audit logs.

2 Peaks and valleys
Queue absorbs a burst of 10,000 orders.

3 Unreliable partner
Retry later instead of failing the user.

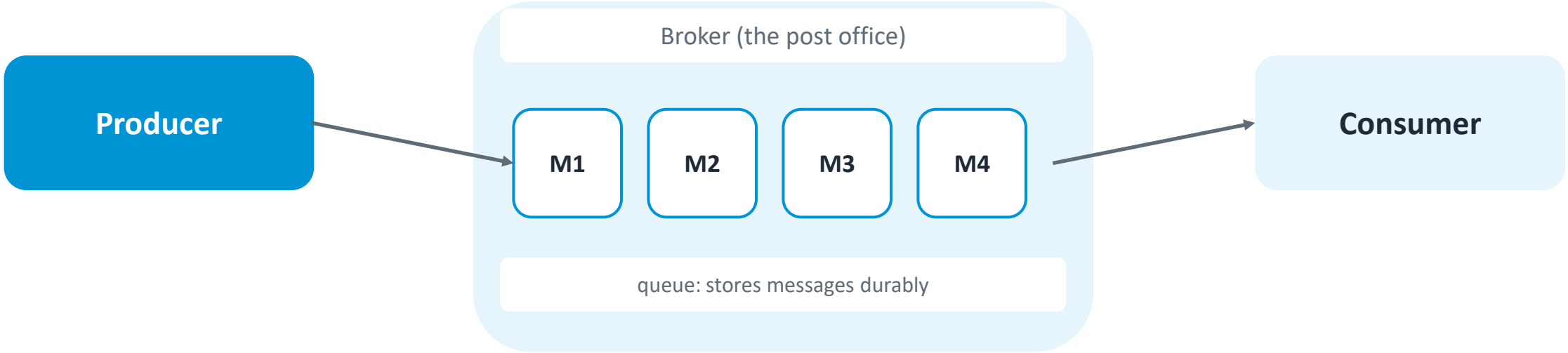
4 Fan-out
One event, many interested services.

5 Long-running work
Report generation, image processing.

6 Not a fit
User needs the answer in this request.

Ask: does the caller need the result in this very request?

The Message Broker



- **Stores:** messages survive restarts (persistent).
- **Routes:** queue, topic, subscription.
- **Tracks:** who got what, redelivery, expiry.
- **Examples:** Apache ActiveMQ Artemis, Azure Service Bus, RabbitMQ.

JMS Concepts at a Glance

1 ConnectionFactory
Creates connections to the broker.

2 Connection
One network link; heavy, share it.

3 Session
Single-threaded context; can be transacted.

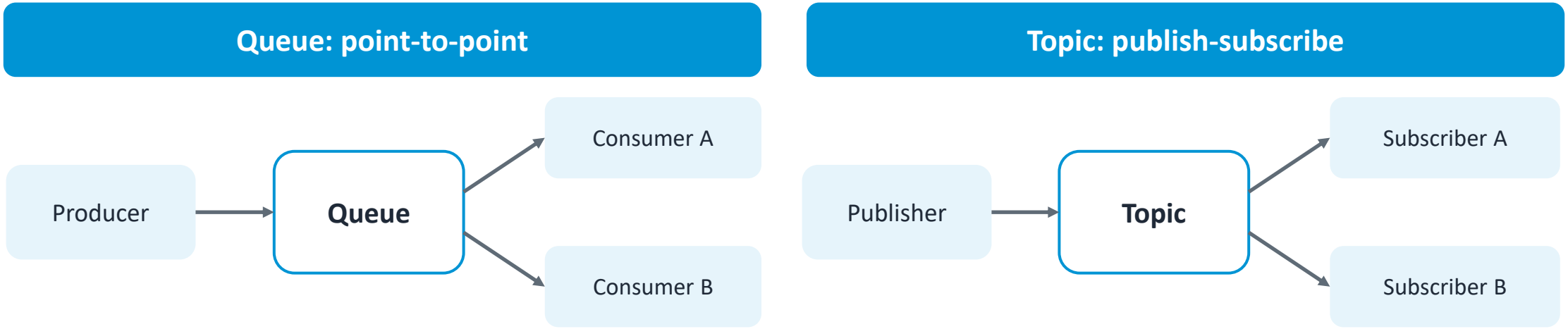
4 Destination
Queue or Topic: the address.

5 Producer
Sends messages to a destination.

6 Consumer
Receives messages from a destination.

JMS = Jakarta Messaging: a Java API, not a wire protocol

Queue vs Topic



- **Queue:** each message goes to exactly ONE consumer; consumers share the work.
- **Topic:** each message goes to EVERY subscriber.
- **Durable subscription:** a subscriber that was offline still gets its copies.

Anatomy of a JMS Message

Headers
JMSMessageID, JMSCorrelationID, JMSDestination, JMSTimestamp, JMSDeliveryMode

Properties
your own key/values: businessId, _type

Body
text (JSON), bytes, map, object

```
jmsTemplate.convertAndSend("learnly.enrollment.events", event, m -> {
    // header: link request and reply / trace
    m.setJMSCorrelationID(event.eventId());
    // property: filterable, readable by consumers
    m.setStringProperty("businessId", event.eventId());
    return m;
});
// JMSMessageID is assigned by the broker
// when the message is sent: you cannot set it
```

Business identity belongs in your own property, not in JMSMessageID

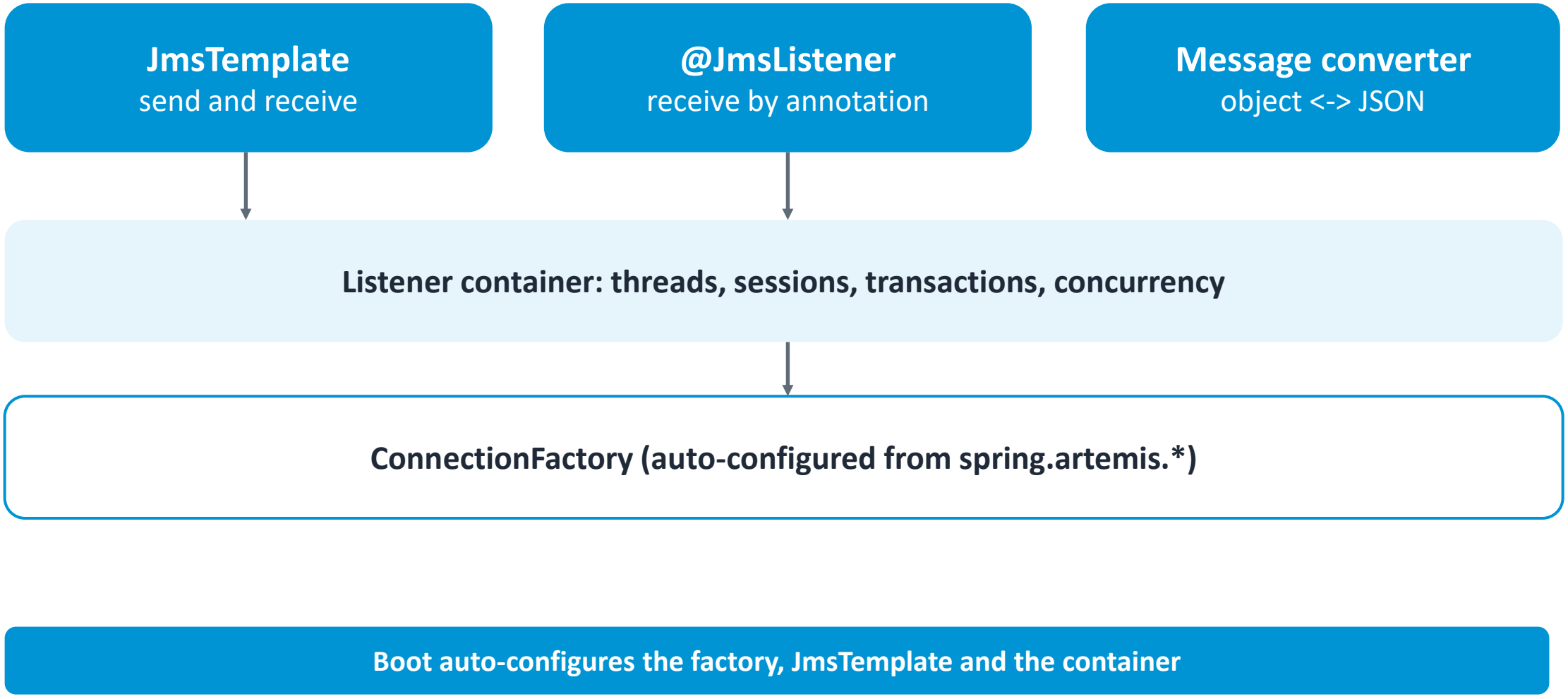
The Plain JMS API in Code

- **Verbose:** create, start, close everything.
- **Checked exceptions:** JMSException everywhere.
- **Resources:** connection and session must be closed.
- **Spring removes** this boilerplate (next slides).

```
try (Connection con = factory.createConnection();
     Session session = con.createSession(
         false, Session.AUTO_ACKNOWLEDGE)) {
    Queue queue = session.createQueue("learnly.enrollment.events");
    MessageProducer producer =
        session.createProducer(queue);
    TextMessage msg =
        session.createTextMessage("{\"id\":1}");
    producer.send(msg);
}
```

jakarta.jms.* in Spring Boot 4 (not javax.jms)

Spring JMS Building Blocks



Spring Boot and Artemis Setup

```
spring.artemis.mode=native
spring.artemis.broker-url=${ARTEMIS_URL:tcp://localhost:61616}
spring.artemis.user=${ARTEMIS_USER:artemis}
spring.artemis.password=${ARTEMIS_PASSWORD:artemis}
spring.jms.listener.concurrency=1
spring.jms.template.receive-timeout=2s
```

- **starter:** spring-boot-starter-artemis.
- **mode native:** remote broker (not embedded).
- **URL options:** tcp://host:61616?consumerWindowSize=0
- **Ports:** 61616 messaging, 8161 web console.

Credentials come from environment variables, with local defaults

Sending with JmsTemplate

- **convertAndSend:** object in, converter makes the message.
- **Destination:** a name; created on first use.
- **Post processor:** set headers and properties.
- **send(...)** gives full control over the Message.
- **receive:** blocking pull; set a timeout!

```
@Service
public class EventPublisher {

    private final JmsTemplate jmsTemplate;

    public EventPublisher(JmsTemplate jmsTemplate) {
        this.jmsTemplate = jmsTemplate;
    }

    public void publish(EnrollmentEvent event) {
        jmsTemplate.convertAndSend(
            "learnly.enrollment.events", event);
    }
}
```

Receiving with @JmsListener

- **Annotation:** binds a method to a queue.
- **Payload:** converted into your record.
- **Raw Message:** optional extra parameter for headers.
- **Return/throw:** normal return = success; exception = failure.
- **Runs on** container threads, not HTTP threads.

```
@Component
public class NotificationListener {

    @JmsListener(destination = "learnly.enrollment.events")
    public void onEnrollment(EnrollmentEvent event,
                             Message raw) throws JMSException {
        log.info("Got {} id={}", event.eventId(),
                 raw.getJMSMessageID());
        mailer.sendWelcome(event.email());
    }
}
```

No @EnableJms needed: Spring Boot enables it

Demo: Send and Receive

What we build

- **JmsTemplate:** sends a welcome request.
- **Broker:** holds it in learnly.enrollment.events.
- **@JmsListener:** receives it, even when it comes late.

How the concept helps

- **Decoupled:** the sender does not wait.
- **Durable:** the broker keeps the letter.
- **Simple:** Boot sets up the connection.

Message Converters

- **Job:** object to message body and back.
- **JSON text:** readable, language neutral.
- **Type id property:** tells the receiver which class to build.
- **Jackson 3:** JacksonJsonMessageConverter in Spring 7.
- **Avoid:** Java serialization (unsafe, tied to Java).

```
@Bean
MessageConverter enrollmentMessageConverter() {
    var converter = new JacksonJsonMessageConverter();
    converter.setTargetType(MessageType.TEXT);
    converter.setTypeIdPropertyName("_type");
    return converter;
}
```

Declare one MessageConverter bean; Boot applies it to JmsTemplate

Demo: JSON Messages and Headers

What we build

- **Converter:** EnrollmentEvent as JSON text.
- **Correlation id:** a header we set.
- **businessId:** our own property.

How the concept helps

- **Readable:** language neutral bodies.
- **Identity:** business id, not JMSMessageID.
- **Typed:** the listener gets a record.

Retry and Max Delivery Count




```
<!-- Artemis broker.xml -->
<address-setting match="learnly.#">
  <dead-letter-address>DLQ</dead-letter-address>
  <max-delivery-attempts>3</max-delivery-attempts>
  <redelivery-delay>0</redelivery-delay>
</address-setting>
```

- **Artemis default:** 10 attempts.
- **Service Bus:** MaxDeliveryCount, default 10.
- **Delay:** redelivery-delay / back-off avoids hammering.
- **Transient errors** deserve retries; permanent ones do not.

Dead-Letter Queues

Artemis

DLQ address and queue you define (DLQ).

Message is a copy with extra properties:

_AMQ_ORIG_ADDRESS, _AMQ_ORIG_QUEUE, _AMQ_ORIG_MESSAGE_ID.

No failure reason is stored: use your logs.

Azure Service Bus

Built-in sub-queue per queue or subscription: <entity>/$deadletterqueue.

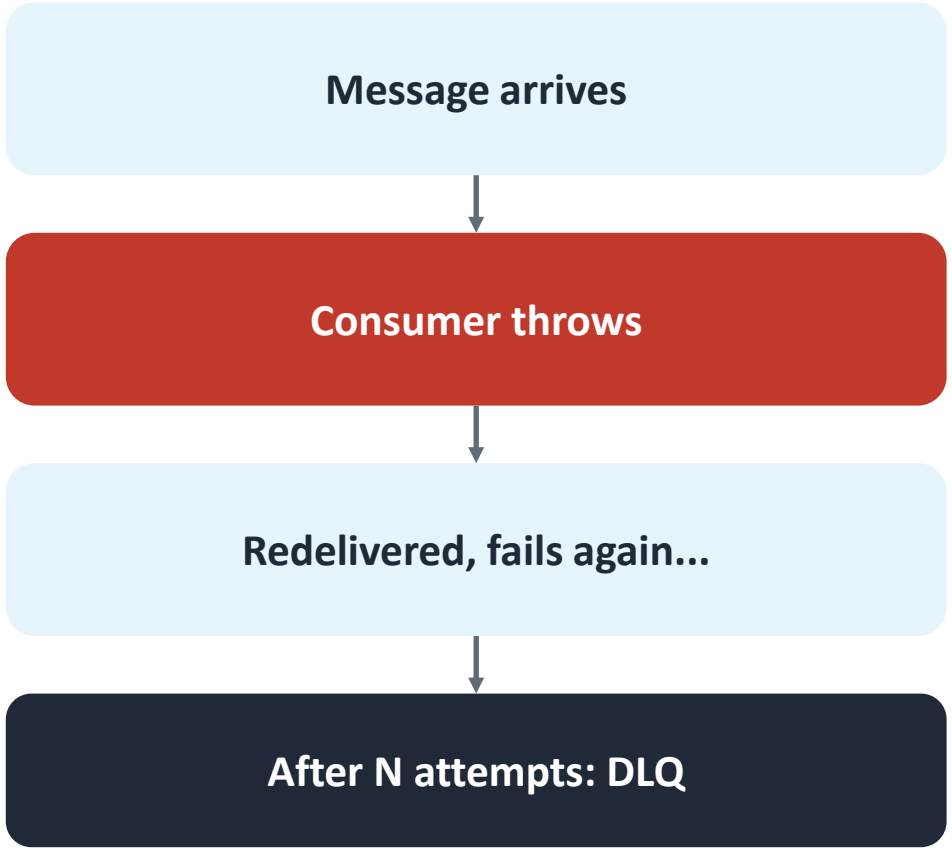
Properties DeadLetterReason and DeadLetterErrorDescription.

Also used for TTL expiry (optional) and filter errors.

- **Purpose:** park what cannot be processed so others keep flowing.
- **A DLQ is not a trash can:** monitor it and alert on depth.

Poison Messages

- **Poison message:** a message that makes the consumer fail every time.
- **Causes:** bad JSON, missing data, a bug, a deleted record.
- **Without limits:** retry forever, block the queue, burn CPU.
- **Detect:** delivery count rising, same id in logs, DLQ depth.



Demo: Retry, Dead Letters and Reprocessing

What we build

- **Poison event:** fails three times.
- **DLQ:** parked with its original address.
- **Resubmit:** fix it and send it back.

How the concept helps

- **Flow:** other messages keep moving.
- **Not a trash can:** inspect the DLQ.
- **Replay:** safe when consumers are idempotent.

Delivery Guarantees

At-most-once

Send and forget; no retry.

Never duplicated, may be lost.

Receive-and-delete, non-persistent.

At-least-once

Retry until acknowledged.

Never lost, may be duplicated.

The normal mode of JMS and Service Bus.

Exactly-once

Effect happens once.

Only by at-least-once plus an idempotent consumer.

- **Why duplicates:** consumer processes, then crashes before the ack.
- **So:** design every consumer for duplicates.

'Exactly-once delivery' is a goal you build, not a switch you set

Duplicates and Idempotent Consumers

- **Idempotent:** doing it twice = doing it once.
- **Key:** business id sent by the producer.
- **Table:** processed_message with a UNIQUE key.
- **Same transaction:** insert the key and the result together.
- **Not JMSMessageID:** a re-send gets a new one.

```
@Transactional
public boolean processOnce(String businessId,
                           EnrollmentEvent event) {
    if (messages.existsByMessageId(businessId)) {
        return false;          // duplicate: skip
    }
    messages.saveAndFlush(
        new ProcessedMessage(businessId));
    enrollments.save(toEntity(event));
    return true;
}
```

The UNIQUE constraint is the final judge when threads race

Demo: Duplicates and Idempotent Consumers

What we build

- **Duplicate:** e-103 is sent twice.
- **Two emails:** without protection.
- **ProcessedEvents:** skips a seen business id.

How the concept helps

- **At-least-once:** duplicates will happen.
- **Business id:** a re-send gets a new message id.
- **Exactly-once effect:** built, not switched on.

Azure Service Bus Overview

1 **Namespace**
The container: endpoint, tier, security.

2 **Queue**
Point-to-point; one consumer per message.

3 **Topic**
Publish once; copies go to subscriptions.

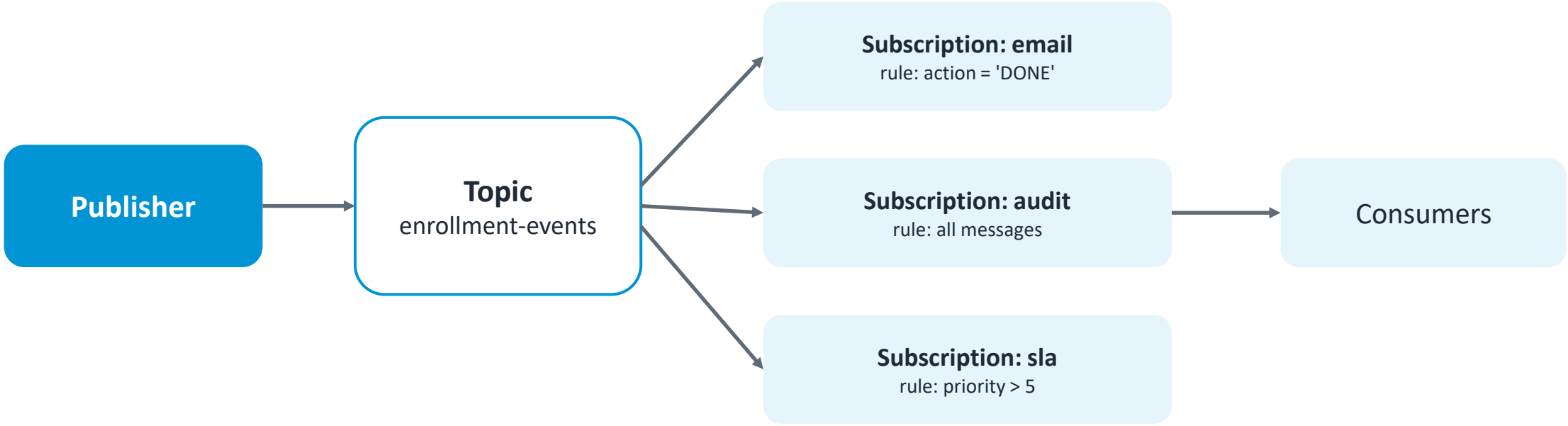
4 **Subscription**
A virtual queue per interested consumer.

5 **Managed**
No servers; Azure runs and patches it.

6 **Tiers**
Basic, Standard, Premium.

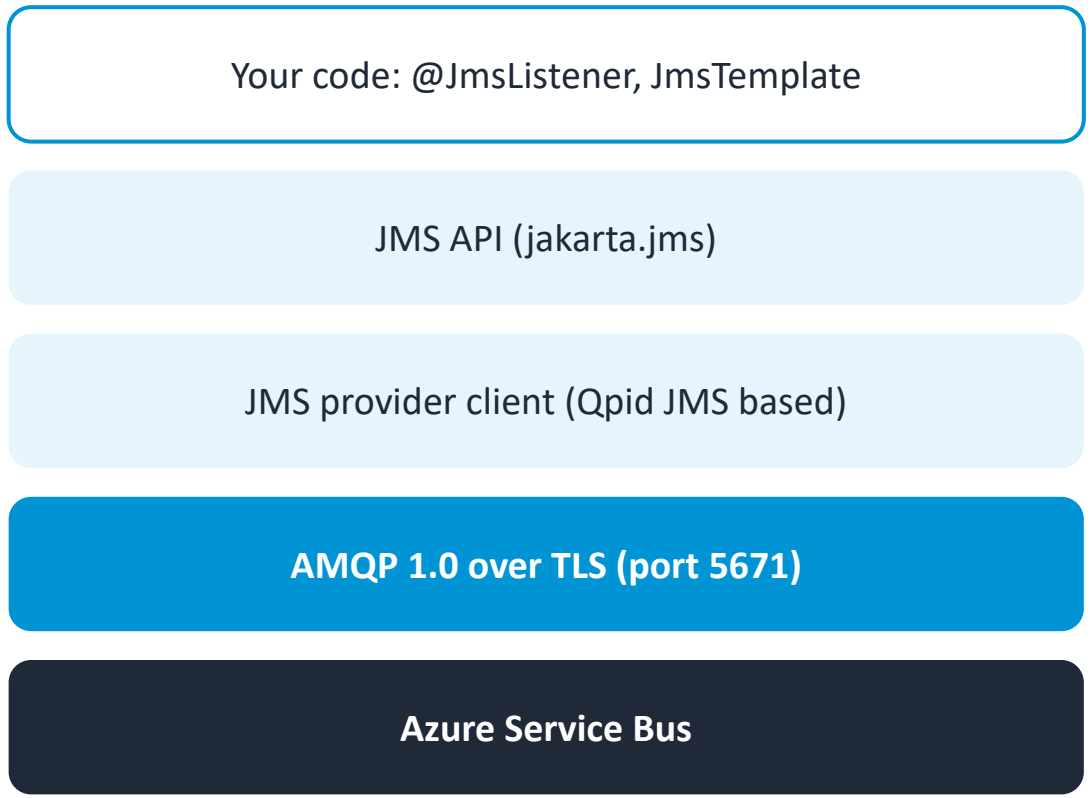
Service Bus = managed enterprise broker, same ideas as Artemis

Queues, Topics and Subscriptions



- **Filters:** each subscription has rules (SQL, correlation).
- **Own copy and own DLQ** per subscription.
- **Same sender,** many independent readers.

AMQP and the JMS Client



- **AMQP 1.0:** open wire standard (ISO/IEC 19464).
- **JMS:** a Java API; AMQP is the protocol underneath.
- **Native SDK:** azure-messaging-servicebus, all features.
- **JMS client:** keeps your JMS code portable.

JMS on Service Bus: Tiers

Premium tier

Supports the JMS 2.0 API. The tier to use for JMS workloads. Dedicated resources.

Standard tier

JMS 1.1 only, reduced feature set. Fine for simple cases.

```
spring:
  jms:
    servicebus:
      connection-string: ${SB_CONNECTION_STRING}
      pricing-tier: premium
```

- **Starter:** spring-cloud-azure-starter-servicebus-jms
- **Prefer:** passwordless (managed identity).
- **New projects:** Microsoft advises the native SDK.

Key Takeaways

1. **Choose wisely**
   Messaging for work that can finish later.

2. **JMS and Spring**
   JmsTemplate, @JmsListener, converter, container.

3. **Service Bus**
   Queues, topics, subscriptions; JMS needs Premium.

4. **Expect duplicates**
   At-least-once plus idempotent consumers.

5. **Plan for failure**
   Max delivery, DLQ, and a reprocessing routine.



Happy Learning :)